Accurate and Efficient Parasitic Capacitance Extraction: A Hybrid Approach Integrating Field Solvers and Deep Neural Networks

Ziheng Li[1], Benyuan Chen[1,], Yuxuan Zhao[1], Juan Li[2], Hui Lv[3], Shaohua Ye[4]*

1 National "111 Research Center" Microelectronics and Integrated Circuits, School of Science, Hubei University of Technology, Wuhan 430068, China

2 University of Science and Technology of China, Hefei 230026, China

3 Hubei University of Education, Wuhan 430205, China

4 University of Chinese Academy of Sciences, Beijing 100049, China

E-mail: shingolala@126.com

Abstract: Accurate parasitic capacitance extraction is critical for IC design, yet existing methods suffer from limited accuracy, efficiency, and generalization. This paper proposes a hybrid approach that integrates a finite element field solver and a deep neural network. Capacitance prediction models for three patterns are constructed to enable end-to-end rapid inference. Using the field solver results as a benchmark, the neural network is validated to achieve relative errors below 1% across all test points. In tests involving 972, 972, and 432 sampling points, the total prediction times are 16.335 s, 33.485 s, and 14.965 s, averaging 27 ms per point.

Key words: Parasitic capacitance; Field solver; Deep neural network; Integrated circuits; Finite element method

1.Introduction

With the continuous scaling of semiconductor technologies, parasitic effects in integrated circuits have become increasingly significant due to shrinking device dimensions and complex interconnects. Among these, parasitic capacitance plays a critical role in determining circuit performance. It aggravates signal delay by forming RC time constants with interconnect resistance [1]; higher capacitance increases propagation delay, thereby reducing timing margins and affecting clock synchronization. It also contributes substantially to power consumption—particularly in CMOS circuits, where dynamic power predominantly arises from charging and discharging parasitic capacitances. Moreover, coupling capacitance between adjacent wires induces crosstalk noise, compromising signal integrity in high-speed and densely routed designs [2]. Consequently, accurate parasitic capacitance extraction has become essential for ensuring circuit correctness and performance in advanced nodes. In modern IC design flows, post-layout parasitic extraction captures the electromagnetic effects of interconnects as parasitic resistors, capacitors, and inductors for subsequent simulation [3]. Among these, capacitance extraction has garnered particular attention due to its direct impact on delay, power, and signal integrity—especially in deep-submicron processes, where complex layout geometries impose

stringent demands on both accuracy and efficiency. Therefore, developing capacitance extraction methods that accelerate computation and reduce resource consumption without sacrificing precision has emerged as a key research focus.

Over the past decades, the continuous scaling of integrated circuits and the increasing complexity of interconnect structures have made accurate parasitic capacitance extraction a critical factor in chip performance analysis. To address this challenge, numerous extraction methods have been developed. Early research focused primarily on field-solving techniques. For instance, Campbell et al. pioneered finite difference method (FDM)-based capacitance extraction by discretizing the spatial domain and solving the Laplace equation to obtain electric field distributions and capacitance matrices [4]. Subsequently, Tausch et al. introduced fast boundary element method (BEM) techniques, leveraging Green's functions to reduce problem dimensionality and significantly improve computational efficiency [5]. Wang et al. applied the method of moments (MoM) to better handle parasitic extraction in complex high-dimensional interconnects [6], while Shabbir et al. proposed Monte Carlo methods to account for floating dummy fills in IC layouts [7]. Yu et al. developed hybrid strategies combining FEM and MoM to balance accuracy and efficiency [8]. More recently, Yu et al. have explored random walk-based capacitance modeling, achieving progress in 3D interconnect simulation [9]. Despite these advances, conventional field solvers still face bottlenecks in computational cost and resource consumption when applied to very large-scale circuits, deep-submicron processes, and 3D integrated structures. Hence, improving extraction efficiency without compromising accuracy remains an active research focus.

Current parasitic capacitance extraction methods can be broadly classified into four categories: empirical formulas, field solvers, pattern matching, and AI-based models. Empirical formulas derive capacitance expressions from theoretical or fitted relationships. Shomalnasab and Zhang decomposed capacitance into parallel-plate, fringe, and lateral components, achieving errors of 2–4% with significantly reduced computation [10]. Bansal et al. employed conformal mapping to analyze fringe capacitance between non-overlapping interconnects in different layers [11]. While fast, empirical approaches struggle with accuracy in dense, multi-layer structures and are best suited for speed-critical applications with moderate precision requirements.

Field solvers compute capacitance by numerically solving electrostatic fields to obtain surface potential distributions. These methods include domain discretization techniques such as FDM [12,13] and FEM [14]; boundary integral equation methods like MoM [15], indirect BEM [16], and direct BEM [17,18]; semi-analytical approaches that combine analytical formulas with numerical corrections [19]; and random walk methods based on statistical simulation [9]. FDM and FEM discretize the entire 3D domain, yielding large linear systems that are accurate but

computationally intensive. BEM discretizes only boundaries, producing smaller systems with faster solution times, though handling internal charge distributions can be challenging. Semi-analytical methods balance efficiency and accuracy but may require additional numerical techniques for complex geometries. Random walk methods offer intuitive implementation and high efficiency for complex structures, but achieving accurate results demands extensive sampling. While all field solvers deliver high precision, they entail substantial computational overhead in terms of time and memory.

To further enhance efficiency and address increasingly complex interconnects, researchers have introduced pattern matching [8] and AI-based techniques [20,21] as alternatives that better balance accuracy and computational cost. Pattern matching precomputes capacitance values for standard interconnect configurations and stores them in lookup tables or libraries. During extraction, new structures are matched against the library to retrieve capacitance values directly, bypassing repeated modeling and simulation. This approach offers high query speed and simplicity, particularly for designs with repetitive patterns and stable process conditions. However, as design scales and process technologies evolve, the diversity of interconnect structures grows rapidly, leading to library dimensionality explosion and coverage gaps—pattern matching fails when encountering unseen configurations.

In contrast, AI-based methods demonstrate superior generalization and flexibility. The core idea is to train machine learning models, such as neural networks [22], to learn the mapping between structural parameters and parasitic capacitance directly, circumventing laborious physical modeling and numerical solution. Kasai et al. employed deep neural networks to model parasitic capacitance in various 3D integrated structures, achieving excellent prediction accuracy and computational speed [23]. Yu et al. introduced convolutional neural networks to automatically extract features from interconnect layouts, further enhancing sensitivity to structural details and model generalization [24]. Unlike pattern matching, AI models can handle unseen structures and adapt to new process nodes through incremental training. Nevertheless, their strong dependence on high-quality training data and limited interpretability pose challenges for deployment in EDA workflows where traceability and reliability are paramount.

In summary, parasitic capacitance extraction methods are transitioning from traditional physics-driven field solvers toward data-driven intelligent techniques. Whether pattern matching or deep learning, each approach offers distinct advantages in accuracy, computational cost, applicability, and scalability. Selecting the appropriate method based on specific process backgrounds and design requirements remains a key consideration in EDA tool development and application.

To address the challenges of high computational complexity and resource consumption in

large-scale parasitic capacitance extraction, this paper proposes a dual-pathway solution. First, through physical modeling and mathematical analysis, the finite element method is employed to solve for potential distribution; accuracy and speed are enhanced by optimizing mesh generation and linear system assembly. Second, via structural parameterization and data-driven modeling, the capacitance extraction problem is reframed as a supervised learning task, enabling fast prediction using neural networks.

In terms of methodology, the original three-dimensional interconnect structure is first simplified into a two-dimensional problem through appropriate approximation and dimensionality reduction techniques. A mathematical model for capacitance extraction is then constructed using a field solver, enabling high-precision capacitance computation via programmatic implementation. Subsequently, key geometric and material features are extracted using a parametric structure description method. These structural parameters, along with the capacitance values computed by the field solver, serve as training data for a neural network, which learns the mapping from structural parameters to parasitic capacitance. Finally, the trained neural network model enables batch fast prediction of capacitance for numerous new structures, significantly improving extraction efficiency. To validate the accuracy of the predictive model, a subset of structures is recomputed using the field solver, and the results are compared with the neural network outputs to assess reliability.

2.Finite Element Field Solver Design

To strike a balance between computational accuracy and efficiency, this paper employs the finite element method to model and solve the capacitance extraction problem. Guided by both practical engineering requirements and mathematical principles, a complete solution pipeline is developed—ranging from differential equation formulation to numerical implementation. Key steps include: selecting appropriate basis functions to construct a finite-dimensional approximation space; applying affine transformations to map physical elements onto a reference element for unified integration; and employing Gauss–Legendre quadrature to achieve high-precision computation of element stiffness matrices and load vectors. During the linear system solution phase, a sparse matrix structure is adopted, and convergence control mechanisms are incorporated to optimize computational efficiency, enabling the handling of large-scale problems.

The finite element mesh generation and system matrix assembly are implemented in C++ to enable accurate and efficient capacitance computation. For each conductor, the entire simulation domain is first discretized into triangular elements. The geometric information of all conductors and surrounding dielectric regions is converted into nodes and elements, which are stored in appropriate data structures for subsequent computation. Next, numerical integration is

performed over each triangular element. Local basis functions and affine coordinate transformations are used to construct element stiffness matrices, which are then assembled into the global sparse matrix and load vector. After the system matrix assembly is completed, boundary potential conditions are applied based on the set of nodes lying on conductor surfaces, yielding the final FEM discrete equations. By invoking a sparse matrix solver, the potential distribution at each node is obtained. Subsequently, the normal electric field intensity on each conductor surface is computed from the potential gradient, and surface integration is performed to extract the total charge on each conductor, which is ultimately used to determine the capacitance matrix. In practical implementation, the finite element method demonstrates superior adaptability and accuracy compared to the finite difference method, particularly when handling complex geometric boundaries and heterogeneous material regions. Moreover, although FEM entails greater computational cost than the boundary element method—due to the need to discretize the entire interior of the domain—it offers broader applicability and is not constrained by requirements for homogeneous media or regular boundary conditions.

Figure 1 shows the triangular mesh generated for the simulation domain, which contains four trapezoidal conductors. The mesh is refined in regions with complex electric field distributions—particularly between and around the conductors—where higher accuracy is required. Triangular elements offer excellent geometric flexibility and adapt well to irregular boundaries. The non-uniform meshing strategy concentrates refinement near conductor surfaces and trapezoidal edges, while gradually coarsening away from these critical areas, balancing precision with computational efficiency.

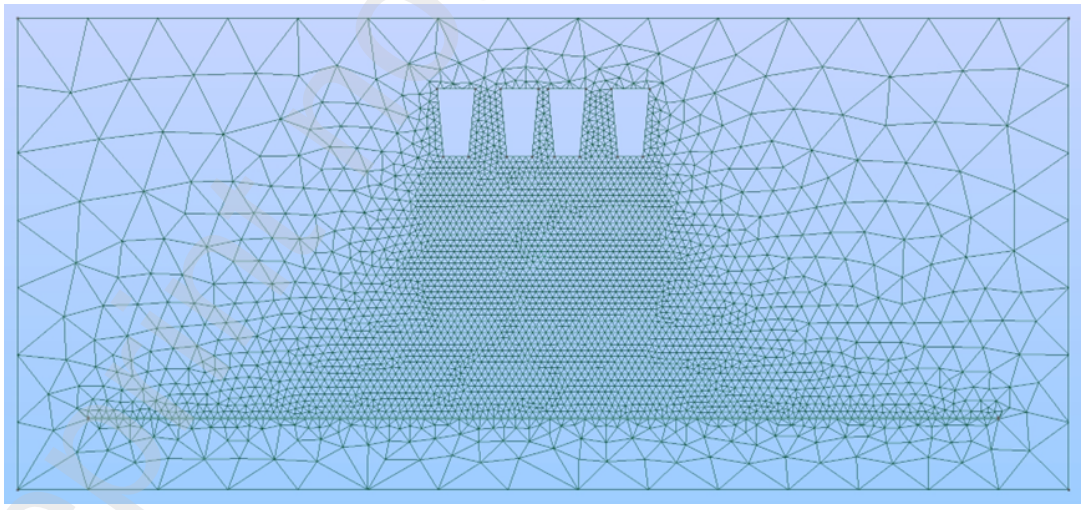

Figure 1. Triangular mesh of the simulation domain

Figure 2 presents the computed potential distribution within the simulation domain. The central region, corresponding to the main conductor, appears in blue and purple, indicating a high potential close to 10 V. As the distance from this conductor increases, the color transitions through green, yellow, and orange, eventually reaching red on the other conductors, where the

potential drops to approximately 0 V. The smooth color gradient demonstrates the capability of the finite element method to accurately resolve subtle spatial variations in the potential field. The equipotential lines extending outward from the high-potential region further illustrate the direction and intensity of the electric field; densely packed lines indicate regions of stronger field strength.

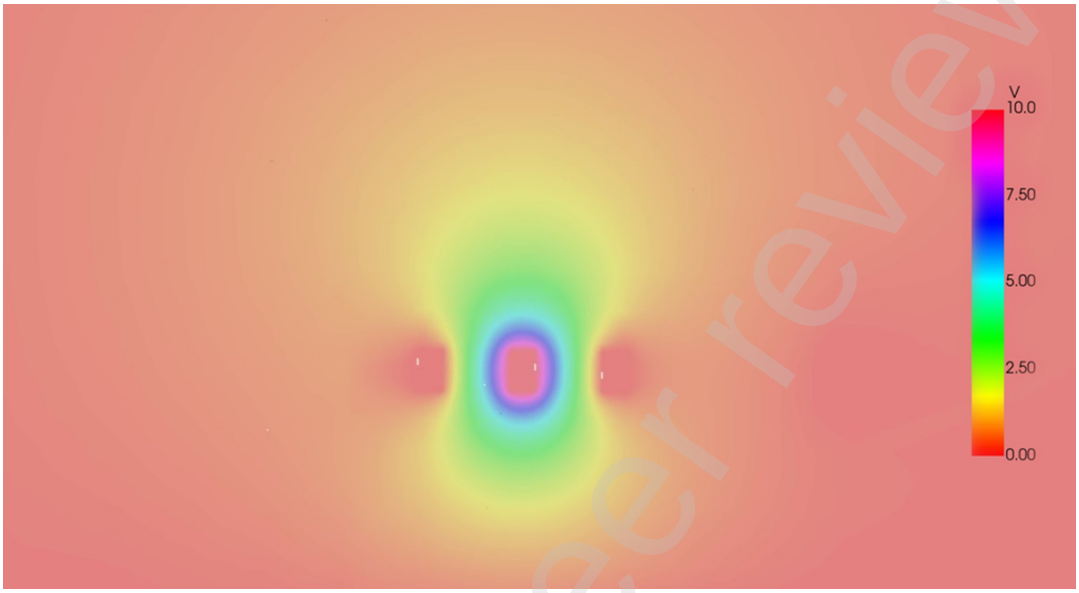

Figure 2. Computed potential distribution of the simulation domain containing four trapezoidal conductors

Tables 1 and 2 summarize the computed capacitance results and corresponding solution times, respectively. As shown, the proposed finite element-based capacitance solver achieves an accuracy within 0.5% of the reference values and computes a full set of capacitance data for a single sampling point in approximately 5 seconds.

Table 1. Computed capacitance results

Capacitance	Reference	Computed
c12	0.050131	0.050013
c13	0.050441	0.050302
c1e	0.002268	0.002376
c1br	0.008154	0.008013
c1bl	0.008799	0.008896
Average error	0.5%	

Table 2. Computation time for selected sampling points

Sampling points	Total computation time

1	5.667 s
2	5.323 s
3	4.887 s
4	5.072 s
5	4.978 s

3.A Deep Neural Network-Based Model for Parasitic Capacitance Prediction

3.1Knowledge-Embedded Strategy Integrating Empirical Formulas

To enhance the accuracy of neural network-based capacitance prediction, empirical formulas from parasitic capacitance extraction can be incorporated into the model design. This is achieved by introducing specialized hidden layers that apply various transformations and feature extraction techniques to the input parameters. By embedding domain knowledge in this manner, the network is better equipped to capture the underlying physical relationships, leading to improved data fitting and overall model performance.

I.Dominant Feature Extraction and Approximation from Empirical Formulas

Empirical formulas for capacitance extraction provide explicit functional relationships between the capacitance of each pattern and its input parameters. Taking the PLATE2L structure shown in Figure 3 as an example, the input parameters include conductor widths w_1 and w_2, spacing *space*, and edge spacing *edgespace*. The metal layer heights *h* for each pattern are also known. The objective is to compute three capacitance components (C_{12}, C_{1b}, C_{1e}) associated with this structure.

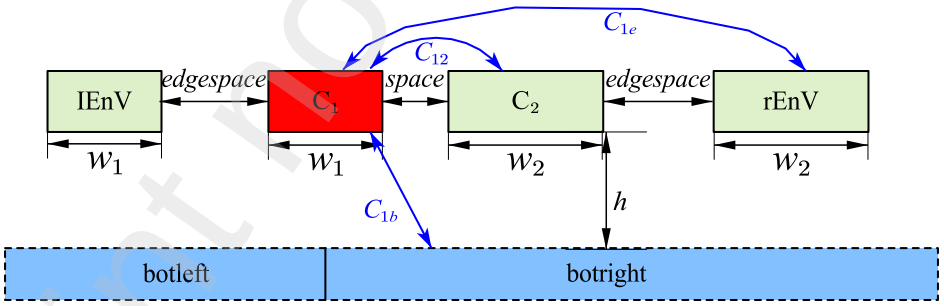

Figure 3. The PLATE2L structure

For the capacitance between C_1 and C_2, both conductors reside in the same layer and thus constitute a lateral capacitance type. The total capacitance consists of two components: overlap capacitance and lateral capacitance. Based on the empirical formula proposed by G. Shomalnasab et al. [10], we have:

$$C_{12} = \frac{\varepsilon_r \varepsilon_0}{\pi} \ln\left(1 + \frac{2w_e}{space}\right) + \varepsilon_r \varepsilon_0 \frac{A}{space} \tag{1}$$

where $\varepsilon_r\varepsilon_0$ is the permittivity, A is the overlap area between C_1 and C_2 and w_e is a constant for a given pattern. Let $a_1=\varepsilon_r\varepsilon_0/\pi$, $a_2=2w_e$, $a_3=A\varepsilon_r\varepsilon_0$, and Equation (1) can then be rewritten as:

$$C_{12} = a_1 \ln\left(1+\frac{a_2}{space}\right)+\frac{a_3}{space} \tag{2}$$

For the capacitance between C_1 and botright (C_{1b}), which falls into the category of conductor-to-substrate capacitance, the empirical formulas from [10][25] give:

$$C_{1b} = \frac{\varepsilon_r\varepsilon_0 A}{h}+\left(\frac{h+t+x}{t}\right)\cdot\frac{2\varepsilon_r\varepsilon_0}{\pi}\ln\left(1+\frac{t}{h}\right) \tag{3}$$

Here, h is the vertical distance from C_1 to the substrate, t is the conductor thickness, and x represents the horizontal distance between metal blocks. Since h is the only variable among these inputs (the others are constant for a given pattern), we can simplify Equation (3) as:

$$C_{1b} = \frac{b_1}{h}+\left(b_2h+b_3\right)\cdot\ln\left(1+\frac{b_4}{h}\right) \tag{4}$$

For the capacitance C_{1e} between C_1 and rEnV, only the lateral capacitance is considered, neglecting any overlap contribution:

$$C_{1e} = \frac{\varepsilon_r\varepsilon_0}{\pi}\ln\left(1+\frac{2w_e}{edgespace}\right) \tag{5}$$

Similarly, this can be expressed as:

$$C_{1e} = c_1\ln\left(1+\frac{c_2}{edgespace}\right) \tag{6}$$

For notational convenience, the parameters *space* and *edgespace* introduced earlier are denoted as s_1 and s_2, respectively.

The Taylor expansion of Equation(2) can be expressed as:

$$C_{12} = \left(a_1\sum_{i=1}^{n}(-1)^{n+1}\cdot\frac{{a_2}^n}{{s_1}^n}\right)+\frac{a_3}{s_1}=\sum_{i=1}^{n}a_i\cdot\frac{1}{{s_1}^n} \tag{7}$$

where a_i (i=1,2,3,…) are the polynomial coefficients. When $s_1\to\infty$, the terms beyond the second order tend to zero and can be neglected.

The relative error of capacitance for each sampling point is defined as:

$$\text{average weight erro} = \frac{\sum_{i=1}^{n}\left|c_i-c_i^*\right|}{\sum_{i=1}^{n}\left|c_i^*\right|} \tag{8}$$

where c_i^* is the reference capacitance value and c_i is the computed value. Equation (8) indicates that for a given absolute error, the relative error becomes larger when the reference capacitance value is smaller. Since capacitance decreases with increasing distance from the main conductor,

particular attention should be paid to prediction accuracy when the parameters s_1, s_2, and h take larger values.

Since s_1 (i.e., space) in this paper takes 15 discrete values ranging from 0.0225 to 4.5, for large values of s_1, based on the analysis of Equation (7), the capacitance C_{12} can be approximated as:

$$C_{12} \approx a_1 \cdot \frac{1}{s_1} + a_2 \cdot \frac{1}{s_1^2} \tag{9}$$

Note that the coefficients a_1 and a_2 in Equation (9) represent polynomial coefficients obtained through fitting and are distinct from the coefficients a_1, a_2 appearing in Equation (7).

Following the same rationale, capacitances between other conductors and the main conductor can also be approximated as linear combinations of the inverse first and second powers of the dominant influencing parameters, in a form analogous to Equation (9). It should be emphasized that this approximation is valid only when the distances between conductors are relatively large. However, as these are precisely the scenarios where prediction accuracy is of greatest concern in this study, the approximation is both appropriate and sufficient for our purposes.

II.Hidden Layer Configuration and Activation Function Selection

Based on the empirical formulas and approximate expressions for capacitance calculation, three types of transformation layers are introduced: linear_layer, power_layer, and log_layer. The linear transformation captures the relationship between conductor width and capacitance; the power transformation models the inverse proportionality of capacitance to the conductor spacing and to the square of conductor spacing; and the logarithmic transformation accounts for logarithmic dependencies present in the empirical formulas.

These transformation layers are incorporated into the hidden layers of the neural network architecture. The network autonomously learns both linear and nonlinear relationships from the input data, fitting the optimal coefficients for the expressions, and subsequently processes the transformed features through additional layers to produce the final capacitance output. This design enables the model to achieve higher representational accuracy compared to a network composed solely of fully connected layers. Table 3 presents the time required for different layer configurations to converge to within 1% error. Among the configurations evaluated, the combination of fully connected layers, negative first-power transformation, and batch normalization achieves the fastest convergence speed, demonstrating that batch normalization significantly accelerates convergence.

Table 3. The convergence time required for various hidden layer configurations

	FC	Power(-1)	FC	FC	FC	FC

Layer Configuration			power(-1)	power(-1) power(-2)	log	power(-1) BN
Convergence Time	60s	30s	20s	25s	40s	8s

In Table 3, "FC" denotes fully connected layers, "Power(-1)" and "Power(-2)" refer to transformations using the inverse first and second powers of the input parameters, respectively, "log" represents logarithmic transformation, and "BN" stands for batch normalization. After evaluating linear, power, and logarithmic transformations, it was observed that the logarithmic transformation yielded relatively poor accuracy. Consequently, the logarithmic transformation is excluded from the DNN architecture discussed in the following sections.

3.2DNN Architecture and Training Methodology

In this paper, both fully connected and residual architectures are employed to train the DNN model for capacitance prediction. This section introduces the various network structures explored during the training process, compares their performance, and discusses the respective advantages, disadvantages, and functional roles of each configuration.

I.Fully Connected Neural Network Architecture Design

For the PLATE2L pattern, considering variations in metal layer heights, five structured parameters serve as inputs: w_1, w_2, *space*, *edgespace*, and *h*. The network outputs three capacitance values: C_{12}, C_{1b}, and C_{1e}. Accordingly, the input layer of the DNN contains 5 neurons, and the output layer contains 3 neurons. Experimental evaluation revealed that a hidden layer size of 20 neurons yields optimal training performance. The resulting fully connected architecture, designated as noole, is illustrated in Figure 4. The network begins with 5 input neurons fully connected to a hidden layer of 20 neurons, followed by a SELU activation function. This is followed by another fully connected layer with 20 neurons, again activated by SELU. Finally, the 20 neurons are fully connected to the 3 output neurons.

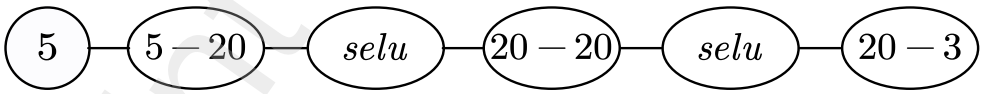

Figure 4. Architecture of the noole network

II.Residual Network Architectures

Building upon the analysis in Section I, power transformations are incorporated into the neural network architecture. To preserve both linear and nonlinear transformations simultaneously, residual structures [26] are adopted, as illustrated in Figure 5. In this figure, the fully connected layers (5→20 and 20→20) perform linear transformations on the input data. "Power" denotes power-law transformation layers, where "power(-1)" applies the inverse first power and "power(-2)" applies the inverse second power to the input parameters. "BN" stands

for batch normalization, which normalizes the output of the preceding layer. The SELU activation function is employed throughout all architectures. The super and zero structures combine the outputs of a linear transformation and a power(-1) transformation via element-wise addition, followed by further linear layers and activation functions. The difference lies in the inclusion of batch normalization: the zero structure incorporates BN, while super does not. The hyper and hana structures similarly combine three branches: linear, power(-1), and power(-2) transformations. After element-wise addition, the result is passed through linear layers and activation functions. The hana structure additionally applies batch normalization, whereas hyper does not. The leaf structure applies batch normalization only to the power(-2) branch before merging with the other branches.

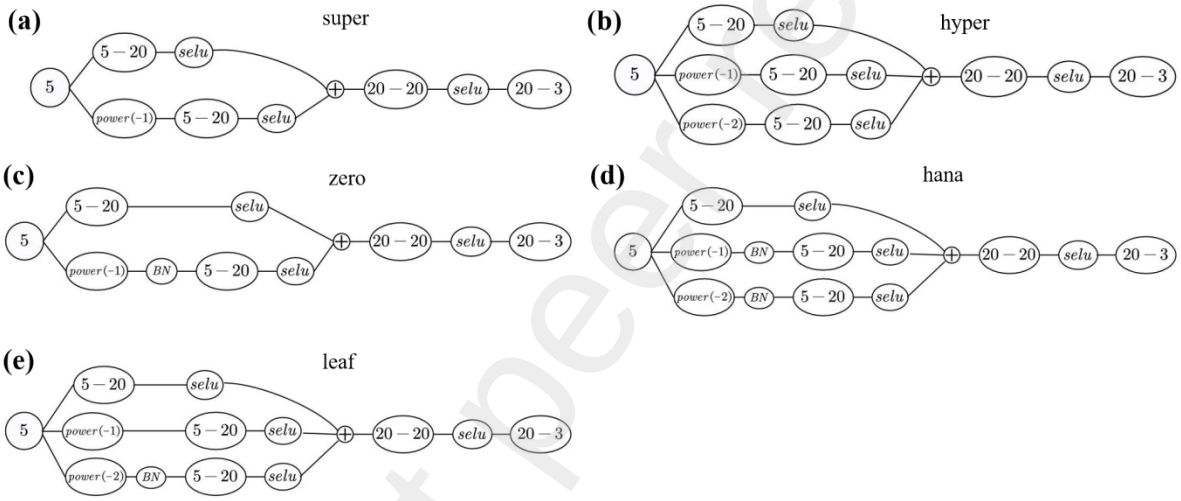

Figure 5. Residual network architectures: (a) super; (b) hyper; (c) zero; (d) hana; (e) leaf

The first pathway processes the input through a simple fully connected layer, serving as a form of shallow information propagation. Even when other pathways apply complex transformations to the input, this pathway ensures that the model retains part of the original input information, helping to prevent the loss of critical features. The remaining pathways apply different nonlinear transformations (e.g., power(-1) and power(-2)) to the input data, enabling the model to extract features from multiple perspectives and scales. Such diversified feature extraction facilitates a better understanding and fitting of complex input data, thereby improving prediction accuracy. By merging the outputs of the three pathways, the model effectively fuses the features extracted from different branches [27]. This merging operation both preserves the original input information and enhances the representational capacity of the model, enabling it to leverage a combination of diverse features for prediction.

Prior to neural network training, the input data are preprocessed using standardization (std), which transforms the data to have zero mean and unit variance. This scaling of all features to a similar range facilitates faster convergence of the optimization algorithm and accelerates the

gradient descent process, enabling the model to reach the optimal solution more efficiently. In this study, the architectures denoted as noole, super, hyper, zero, and hana were trained with such standardization applied to the input data. To investigate the effect of standardization, counterpart models—namely moole, sup, hyp, one, and yoo—were constructed with identical network structures but trained on raw, non-standardized inputs. Beyond the presence or absence of preprocessing, other training parameters—such as the learning content (i.e., the input parameter configurations and corresponding reference capacitance values), the number of samples per training batch, and the learning rate—can also be varied for each architecture.

III.Transfer Learning Mechanism

Transfer learning [28] refers to the process in which, after a model has been trained for a certain number of iterations, the learned features are retained, and the model is fine-tuned on a new dataset with modified training configurations (e.g., learning content, number of samples per batch, learning rate) to meet the specific requirements of a target task. For example, the hana model was initially trained on data from the metal1 layer of the PLATE2L pattern, using 324 samples per batch and a learning rate of 0.001. After 8192 iterations, the model parameters were preserved and transferred to a new model, denoted as sushi. The sushi model shares the same architecture as hana but is further trained on data from three layers (metal1, metal2, and metal3) of PLATE2L with a reduced learning rate of 0.0001.

The inheritance relationships among the models employed in this study are illustrated in Figure 6. In this diagram, boxes represent model names, and arrows indicate the transition process between models, with annotations specifying the modifications applied. The meanings of the abbreviations are listed in Table 4; other abbreviations follow analogous conventions. For instance, the first column in the figure shows that the sup model is derived from the super model by omitting standardization (std) preprocessing. After 1024 iterations of training on sup, the model is adapted to kyle by replacing the input data from a single layer (metal1) with three layers (metal1, metal2, and metal3). The kyle model is then trained for another 1024 iterations, after which the learning rate is reduced from 0.001 to 0.0001, yielding the model kyleloe.

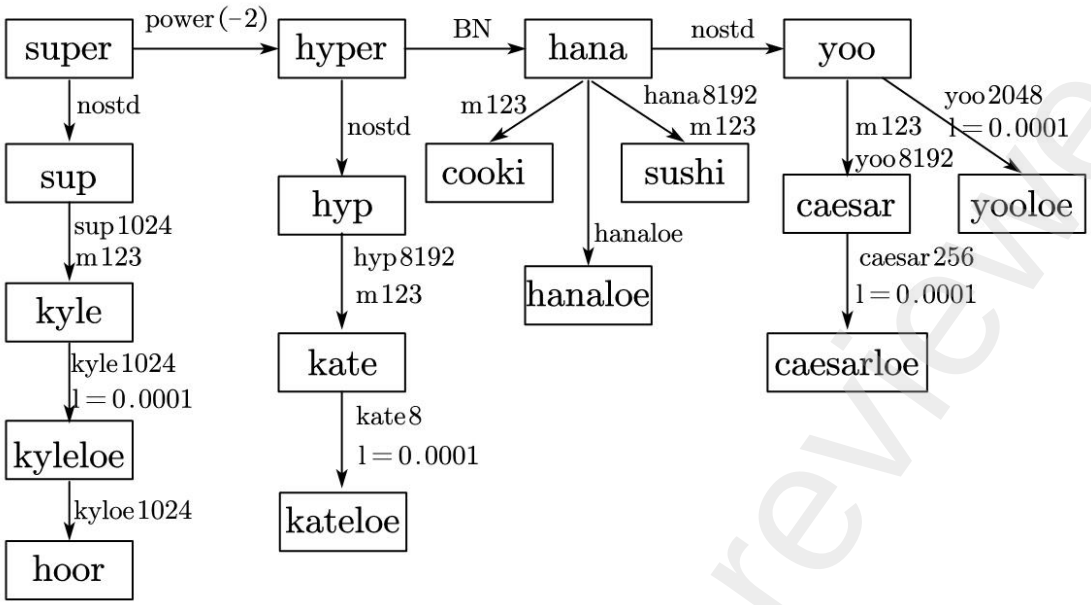

Figure 6. Inheritance diagram of DNN models

Table 4. Description of abbreviations

Abbreviation	Meaning
power(-2)	Inclusion of negative second power transformation
nostd	No standardization applied to input data
BN	Batch normalization applied after nonlinear transformation
l=0.0001	Learning rate set to 0.0001
m123	Training data from metal1, metal2, and metal3 layers
sup1024	Continuation of training based on sup model after 1024 iterations
kyle1024	Continuation of training based on kyle model after 1024 iterations
kate8	Continuation of training based on kate model after 8 iterations

3.3Neural Network Model Training and Evaluation

This section presents an error analysis of all previously introduced models. Similar architectures are grouped together to facilitate a comparative evaluation of individual model performance.

I.Comparison of Minimum Average Errors Across Models

The error for each sampling point is calculated using Equation (8). The average error is obtained by taking the mean of the errors across all sampling points. Figure 7 presents a scatter plot showing the minimum average error achieved by each model, along with the corresponding

maximum error across sampling points. As illustrated, the minimum average error progressively increases from hanaloe to moole. Notably, all models prior to kate achieve minimum average errors below 1%; however, none of these models reduce the maximum sampling point error below the 1% threshold.

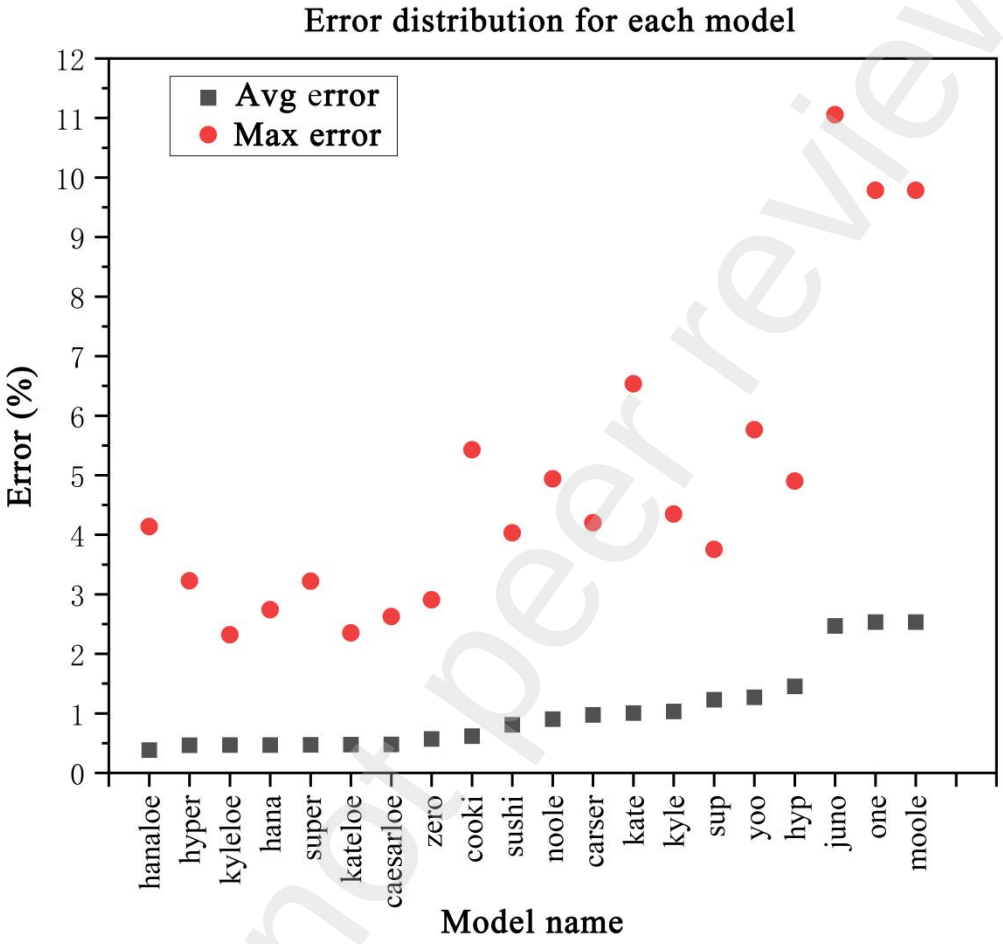

Figure 7. Scatter plot of error distribution for each model

II.Cross-Model Comparison and Generalization Capability Assessment

Figure 8 illustrates the error variation with respect to the number of training iterations for different model configurations. Subfigures (a), (b), (c), and (d) compare models under identical conditions except for data preprocessing, activation functions, power transformations, and batch normalization, respectively.

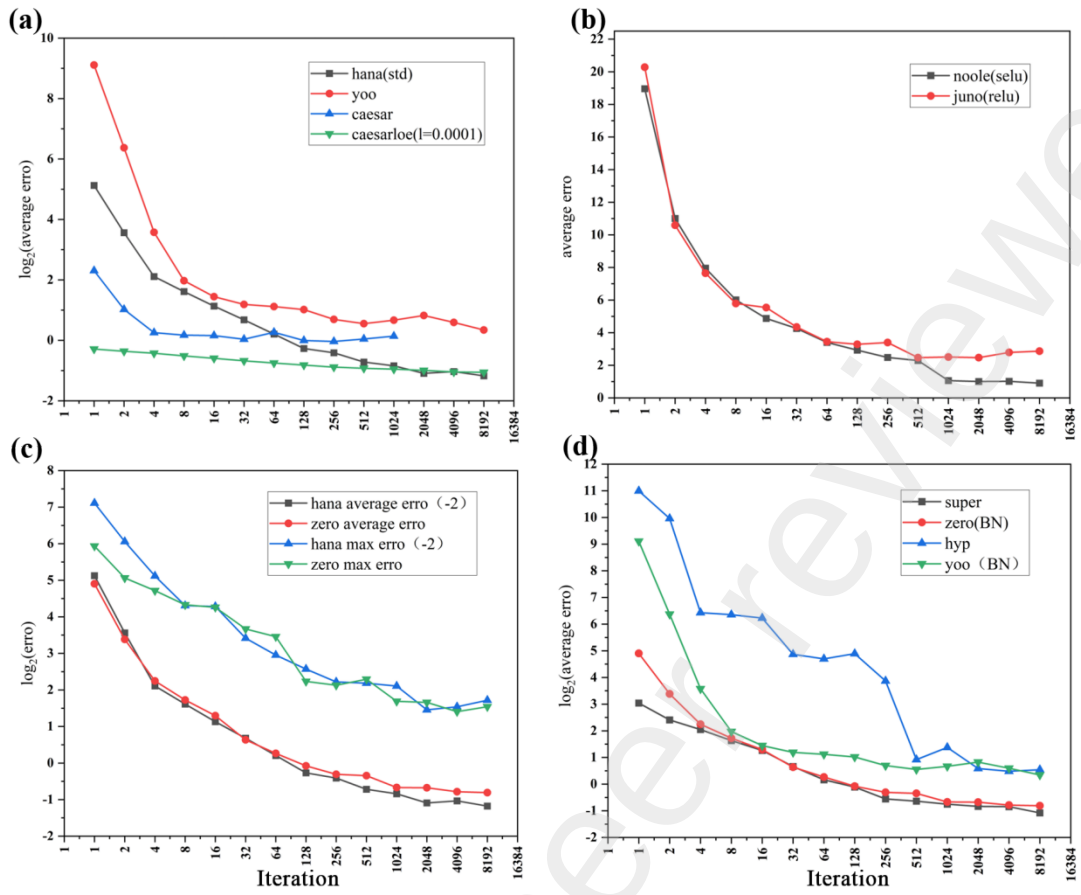

Figure 8. Error vs. number of iterations for different model configurations: (a) with and without standardization (std) preprocessing; (b) comparison of activation functions; (c) comparison of mathematical transformation layers; (d) with and without batch normalization (BN)

In Figure 8(a), the hana model applies standardization (std) preprocessing to the input data prior to training, whereas the other three models are trained on raw, non-standardized inputs. The caesar model continues learning from the yoo model after 8192 iterations by switching to a new dataset; caesarloe further continues from caesar after 256 iterations with the learning rate reduced from 0.001 to 0.0001. Figure 6 illustrates the inheritance relationships among these models. As shown in Figure 8(a), under the same learning rate, models with standardized inputs converge faster, achieve lower errors, and yield better training performance. However, the caesarloe model—which omits standardization but adopts a reduced learning rate of 0.0001—can still converge to an error comparable to that of hana. Given that standardization relies on the statistical properties (particularly the mean and standard deviation) of the original data, avoiding preprocessing may be preferable in certain scenarios. By lowering the learning rate, comparable prediction accuracy can be achieved without standardization, albeit at the cost of significantly slower training.

Figure 8(b) compares the training performance of models using SELU versus ReLU activation functions. As iterations increase, the model employing SELU consistently achieves a lower

average error, demonstrating superior training performance. Therefore, SELU is adopted as the activation function throughout this work.

Figure 8(c) presents the average and maximum errors of models with and without the inclusion of a negative second power transformation in the residual structure. The network architectures of hana and zero are shown in Figure 5. The results indicate that models incorporating the negative second power transformation achieve lower average errors compared to those using only the negative first power transformation. However, the maximum sampling point error tends to be larger in most cases for the former. Since the objective of this study requires that the average error across all sampling points remain below 1%, careful consideration is warranted when selecting the hana model if strict constraints on maximum error are imposed.

Figure 8(d) examines the effect of applying batch normalization after nonlinear transformations. The super and zero structures are analogous, as are the hyp and yoo structures, with the key distinction being the presence or absence of BN. Analysis of the figure reveals that BN accelerates convergence and reduces training time. However, it does not guarantee improved accuracy and is sensitive to batch size; small batch sizes may lead to suboptimal normalization effects.

3.4DNN-Based Capacitance Prediction Application

Based on the comprehensive error analysis of the DNN training models presented in the previous section, several well-performing models are selected for capacitance value prediction across three distinct patterns. The capacitance extraction models and corresponding prediction workflows for each pattern are described in the following sections.

I.The Three Patterns

Figure 9 illustrates three representative interconnect patterns: PLATE2L, PLATE3L, and STACK3L, arranged from top to bottom, respectively. Each pattern comprises three distinct metal layers (columns in the figure) with varying heights. The color mapping indicates the relative permittivity: the main conductor is depicted as a light blue trapezoid, while the surrounding conductors are shown in dark blue. A metal substrate layer exists at the bottom of each pattern but is not visible due to its minimal thickness.

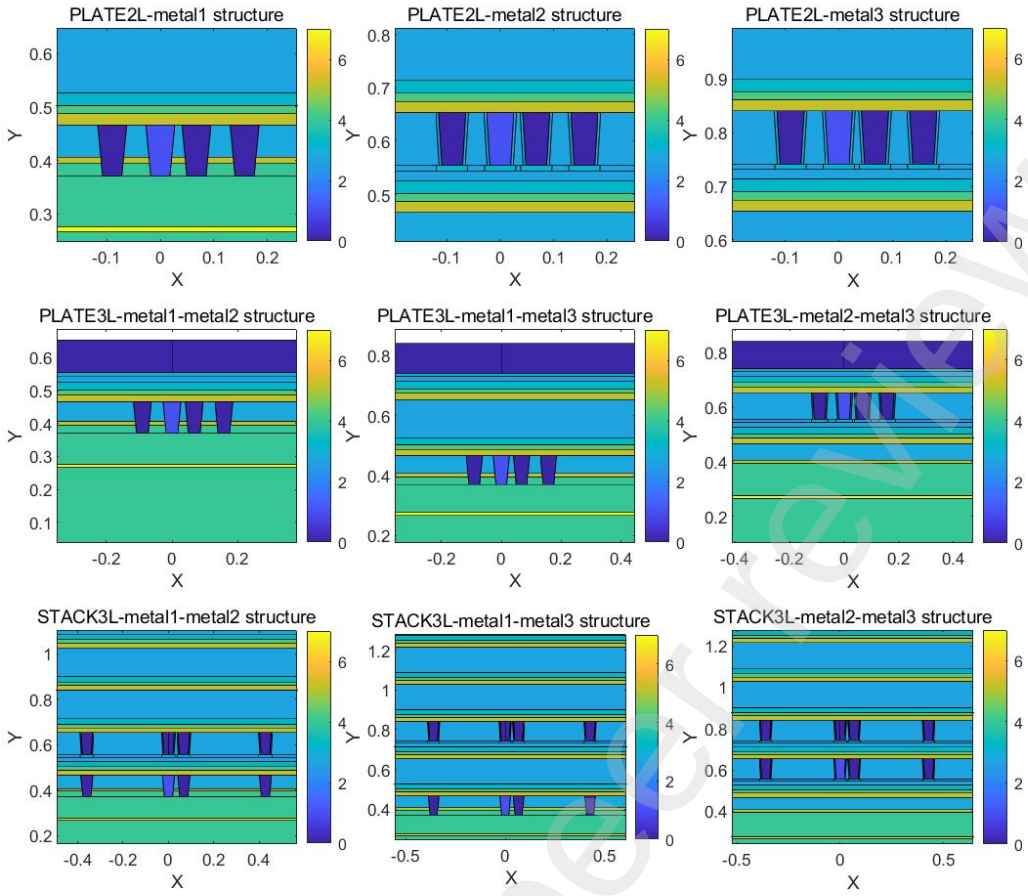

Figure 9. Three typical interconnect patterns

II.PLATE2L Capacitance Prediction Model

As illustrated in Figure 10, for the PLATE2L pattern, three models—caesarloe, kyleloe, and kateloe—achieve average prediction errors below 1%. These models differ in neural network architecture and exhibit varying adaptability to different aspects of the dataset, leading to some variation in the distribution of maximum prediction errors. By taking a linear combination of the capacitance values predicted by the three models, the impact of individual model inaccuracies on the final result can be mitigated; even if one model produces a relatively large deviation, the predictions from the other models can compensate for it.

The design objective requires that the relative error for all sampling points be below 1%. If the sampling point with the largest error meets this criterion, then all points satisfy the requirement. Previous analysis indicates that models incorporating the power(-2) transformation tend to reduce the average error but increase the maximum error. Therefore, a fourth model, denoted as hoor, is introduced. Hoor omits the power(-2) transformation and is derived from the sup model through learning rate reduction and successive transfer learning, thereby achieving high prediction accuracy.

To combine the strengths of the first three models and the hoor model, a dropout-inspired

mechanism [29] is employed. A customized dropout scheme is designed to selectively deactivate certain neurons according to a defined algorithm, effectively choosing between the outputs of the first three models (collectively) and the hoor model. Specifically, if the result from the first three models does not satisfy the error requirement for a given sampling point, the output of the hoor model is adopted as the final prediction.

Based on the above analysis, the neural network prediction framework for the PLATE2L pattern is depicted in Figure 10.

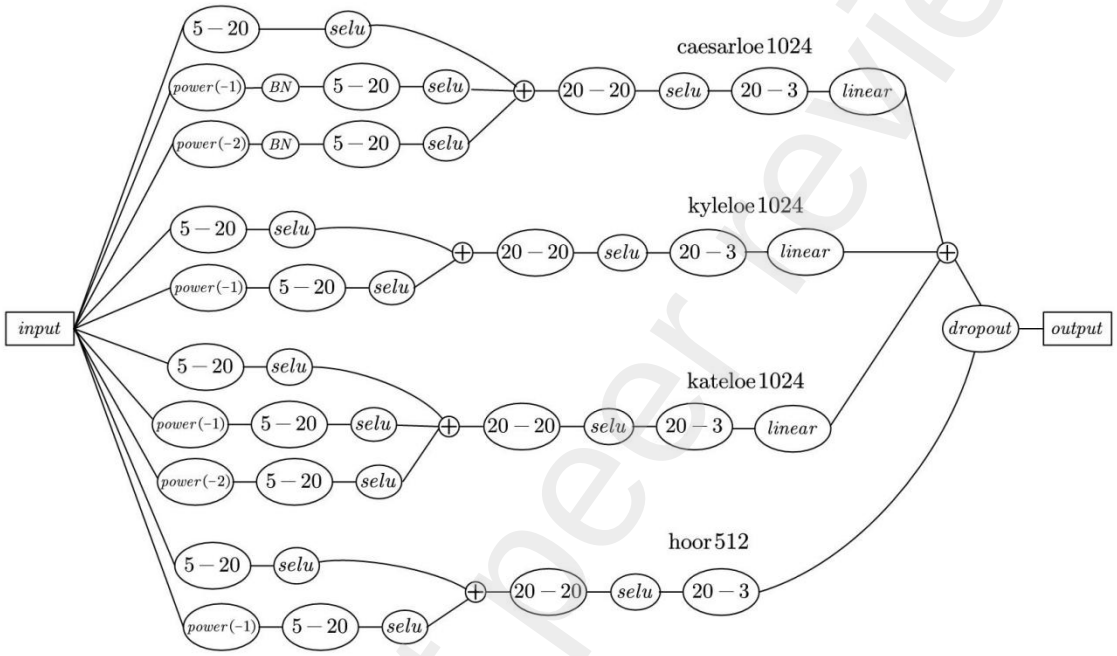

Figure 10. Neural network architecture for capacitance extraction of the PLATE2L pattern

III.PLATE3L Capacitance Prediction Model

For the PLATE3L pattern, the input structured parameters include conductor widths w_1 and w_2, spacings (*space* and *edgespace*), and vertical distances between layers: h_1 (between layer1 and layer2) and h_2 (between layer2 and layer3). This results in a total of six input parameters, and the network is required to predict five capacitance values. Consequently, the previously used hidden layer size of 20 neurons is no longer sufficient; here, the number of neurons in the hidden layers is increased to 50, with an input layer of 6 neurons and an output layer of 5 neurons.

Adopting architectures analogous to the yoo, sup, and hyp models described in Section 3.2, three models—denoted as juanloe, yuxuanloe, and shaohualoe—are constructed for PLATE3L capacitance prediction. These models are trained with a reduced learning rate of 0.0001. The relationship between these new models and their predecessors is illustrated in Figure 11.

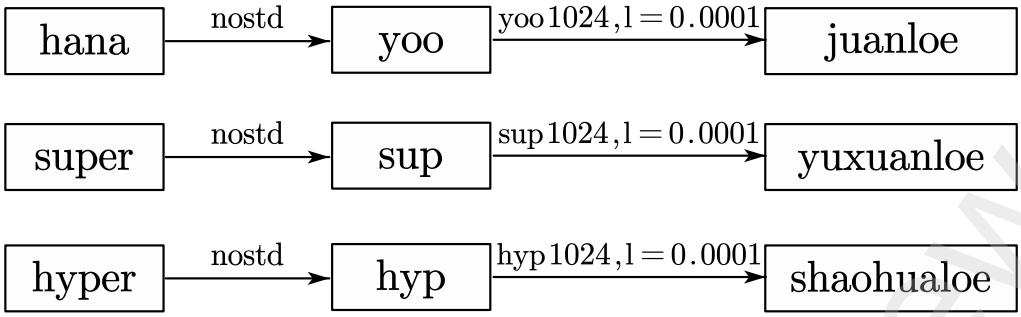

Figure 11. Model diagrams for juanloe, yuxuanloe, and shaohualoe

Building upon the yuxuanloe model, an additional 156 iterations are performed to obtain a fourth model, designated as model_honen. The final neural network architecture for PLATE3L capacitance extraction is depicted in Figure 12.

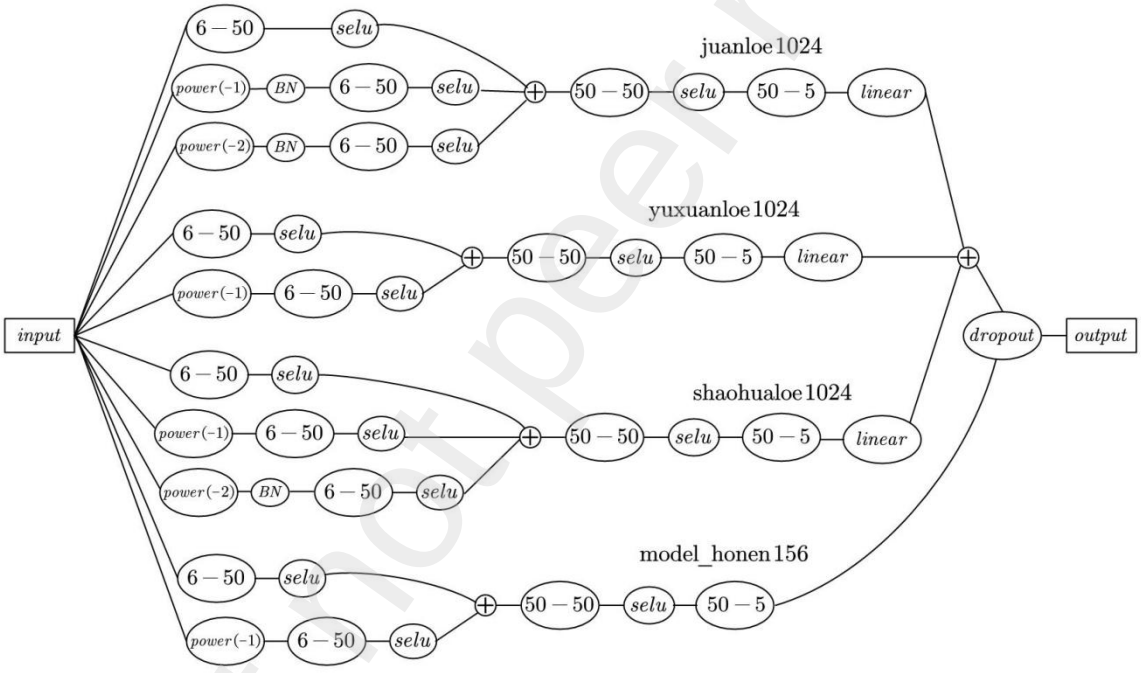

Figure 12. Neural network architecture for capacitance extraction of the PLATE3L pattern

IV.STACK3L Capacitance Prediction Model

For the STACK3L pattern, the input structured parameters include conductor widths w_{Mst} and w_{Wnv}, spacings (space and edgespace), and vertical distances between layers: h_1 (between layer1 and layer2) and h_2 (between layer2 and layer3). This yields a total of six input parameters, and the network is required to predict seven capacitance values. Accordingly, the hidden layer size is set to 50 neurons, with an input layer of 6 neurons and an output layer of 7 neurons.

Following the architectural principles of the yoo, sup, and hyp models introduced in Section 3.2, three analogous models—denoted as loongloe, nezhaloe, and aobingloe—are constructed for STACK3L capacitance prediction. These models are trained with a reduced learning rate of 0.0001. The relationship between these new models and the architectures presented in Section

3.2 is illustrated in Figure 13.

Figure 14 shows the evolution of the mean and maximum relative errors of the three models over the course of iterations. As can be observed, all three models achieve average errors below 1%, and the maximum error eventually decreases to approximately 1.2%. The average error satisfies the design requirement, indicating satisfactory model performance.

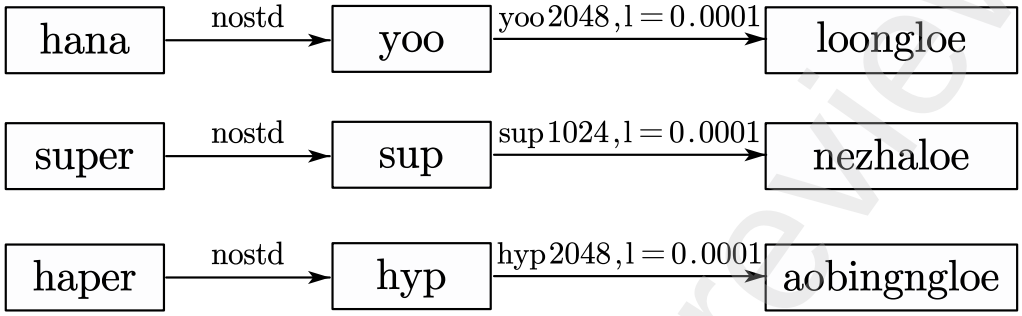

Figure 13. Model diagrams for loongloe, nezhaloe, and aobingloe

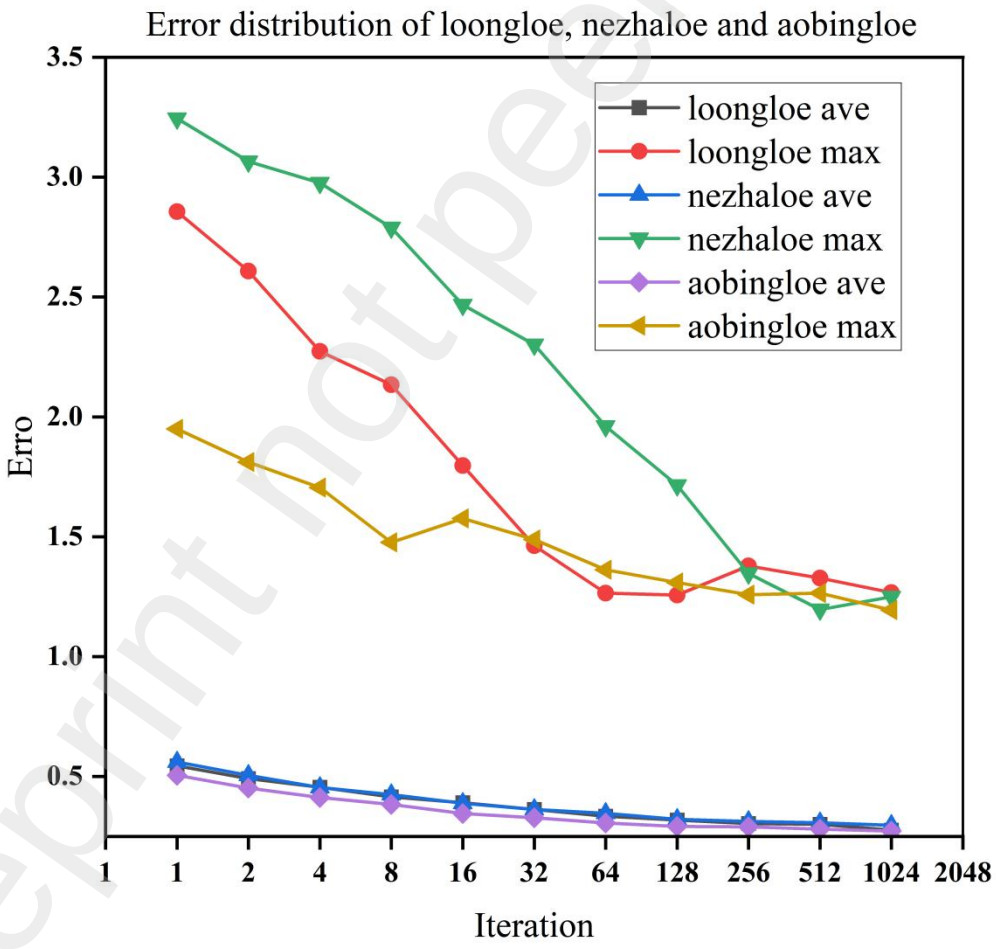

Figure 14. Error vs. number of iterations for loongloe, nezhaloe, and aobingloe

To further enhance prediction robustness, a linear combination of the three models is

adopted. Additionally, the nezhaloe model is iterated for an extra 128 steps to serve as a fourth model, designated as pangu. A dropout-inspired mechanism [29] is then applied to the ensemble comprising the three original models and the pangu model to produce the final capacitance predictions. The final neural network architecture for STACK3L capacitance extraction is depicted in Figure 15. Notably, in the aobingloe model, batch normalization is applied after the negative second power transformation of the input to mitigate the issue of internal covariate shift.

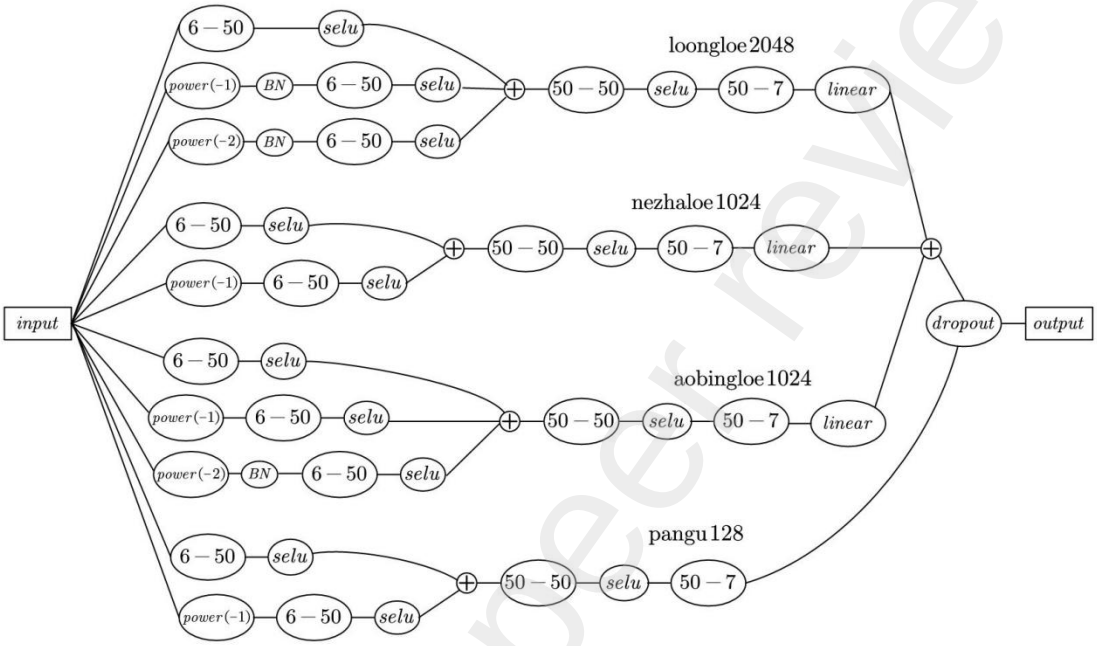

Figure 15. Neural network architecture for capacitance extraction of the STACK3L pattern

V. Capacitance Extraction Workflow

With the neural network architectures for the three interconnect patterns established, a complete capacitance extraction workflow is developed. For a given two-dimensional structure description file and its corresponding sampling point data file, the workflow proceeds as follows: First, feature extraction is performed on the 2D structure file to obtain the conductor widths and spacing parameters. The sampling point data file is then used to train the neural network, generating a trained model file. The extracted feature file, together with the model file, is fed into the prediction model to produce capacitance values for the required sampling points. Finally, by comparing the predicted capacitances with reference capacitance values, the relative errors at each sampling point are computed. The overall capacitance extraction workflow is illustrated in Figure 16.

During feature extraction, the input consists of the geometric distribution and boundaries of the main conductor, surrounding conductors, and dielectric layers. Based on the number of metal layers and the total structure height, the corresponding pattern type is first identified. Subsequently, the conductor widths and spacings are calculated from the vertex coordinates of the conductors, yielding the parameterized structural inputs required for capacitance prediction,

which are then fed into the prediction program.

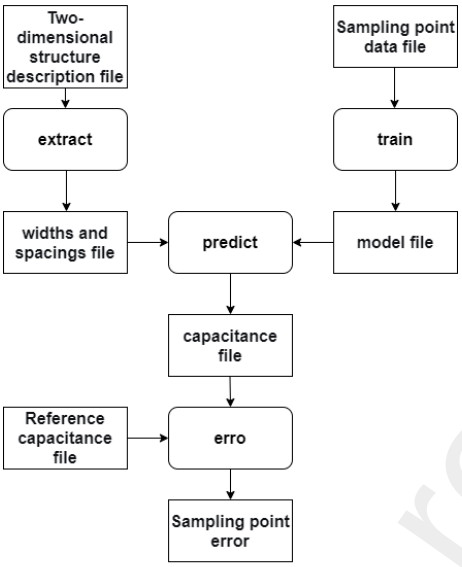

Figure 16. Flowchart of the capacitance extraction process

4.Results

After completing the neural network training, the DNN models are employed to predict capacitance values for a series of new test data, using the capacitance values computed by the field solver as reference standards to validate prediction accuracy and reliability. This section systematically evaluates the prediction performance of the neural network from two perspectives: error analysis and execution time.

During the testing phase, scatter plots of the overall errors for the three patterns are shown in Figures 17, 18, and 19. It can be observed that the prediction errors for all sampling points lie within 1%, meeting the accuracy requirement. The relevant error and time data are summarized in Table 5.

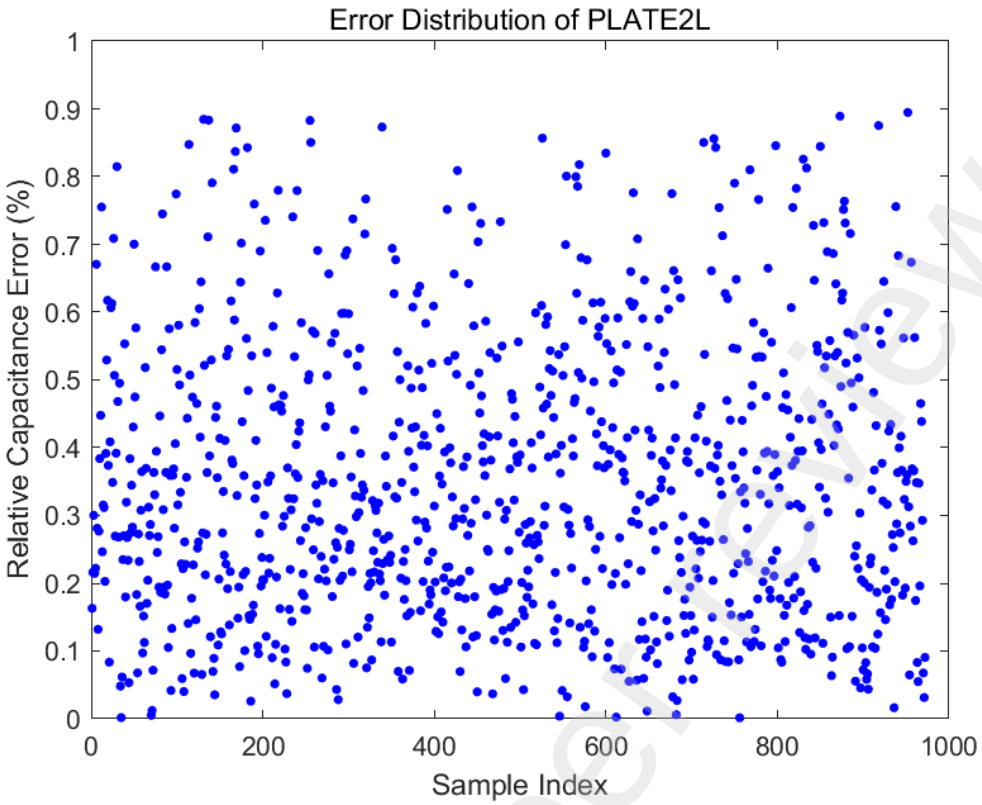

Figure 17. Scatter plot of errors for the PLATE2L pattern

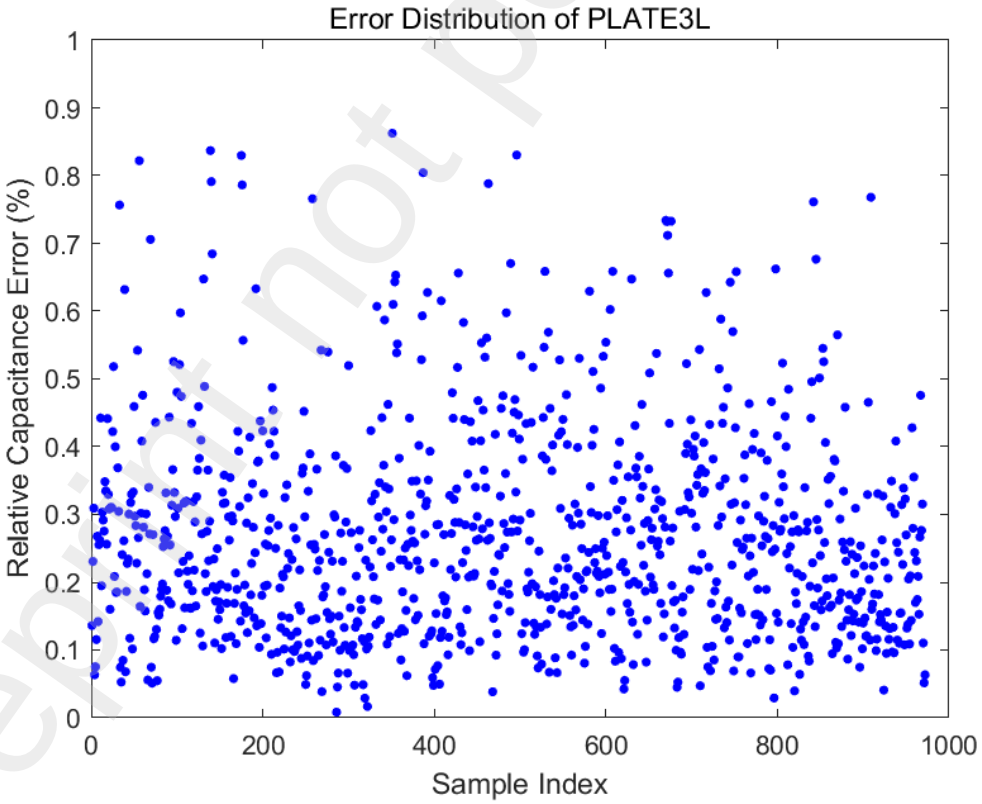

Figure 18. Scatter plot of errors for the PLATE3L pattern

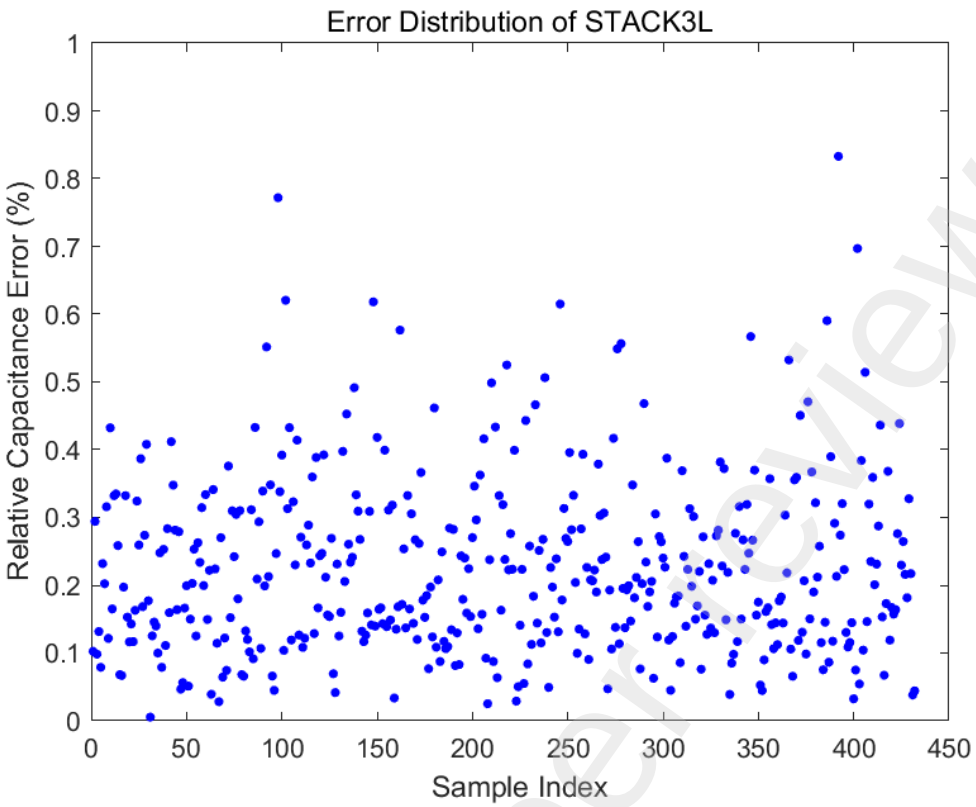

Figure 19. Scatter plot of errors for the STACK3L pattern

Table 5. Error metrics and runtime data

pattern	Average(%)	Min(%)	Max(%)	2sigma(%)	Total time(s)	Peak memory(M)
PLATE2L	0.34652	0.0014358	0.89473	0.40818	16.335	3.09
PLATE3L	0.26527	0.0083017	0.86252	0.30759	33.485	3.19
STACK3L	0.22434	0.0053746	0.83294	0.25789	14.965	2.76

To further analyze the error distribution characteristics of different structures, frequency histograms of the errors (Figure 20) and box plots of the error distributions (Figure 21) for the three initial patterns are presented. As shown in Figure 21, the median error for the PLATE2L structure is approximately 0.35%, with more than half of the sampling point errors concentrated in the 0.2%–0.5% range. For the PLATE3L structure, errors are mainly distributed between 0.15% and 0.35%, with only a very few points exceeding 0.6%. The STACK3L structure exhibits errors predominantly concentrated between 0.13% and 0.30%, with a few slightly higher but not exceeding 0.53%. Overall, the capacitance prediction errors for the vast majority of sampling points across all three structures are well below 1%, confirming the high accuracy and generalization capability of the proposed DNN prediction model.

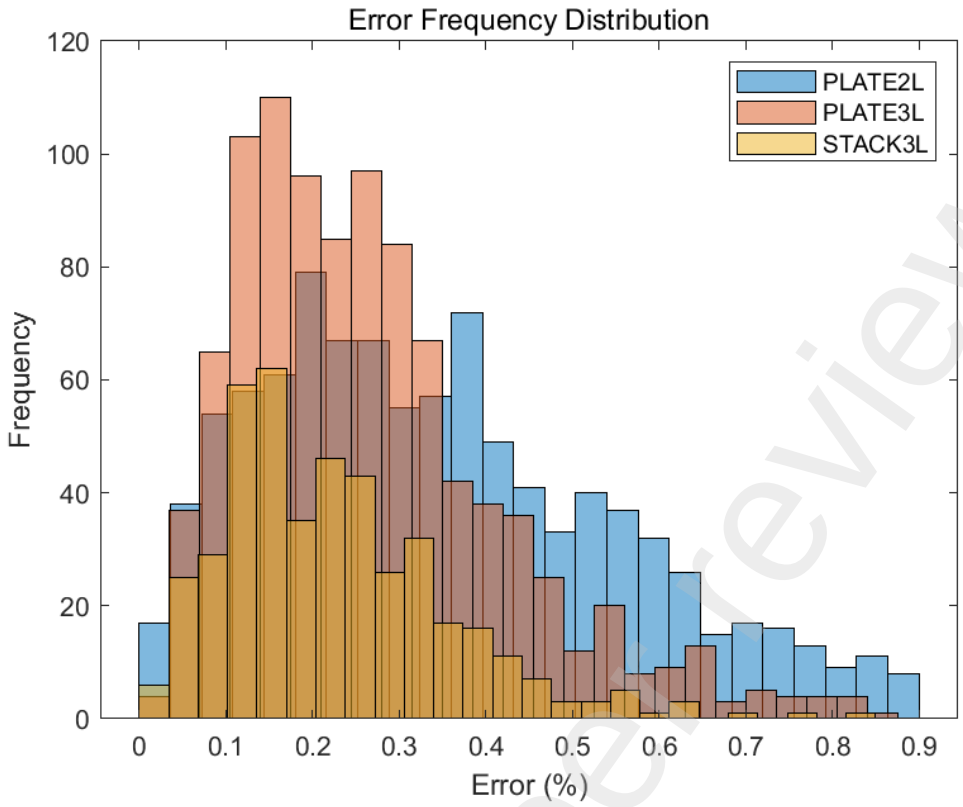

Figure 20. Frequency histograms of error distributions

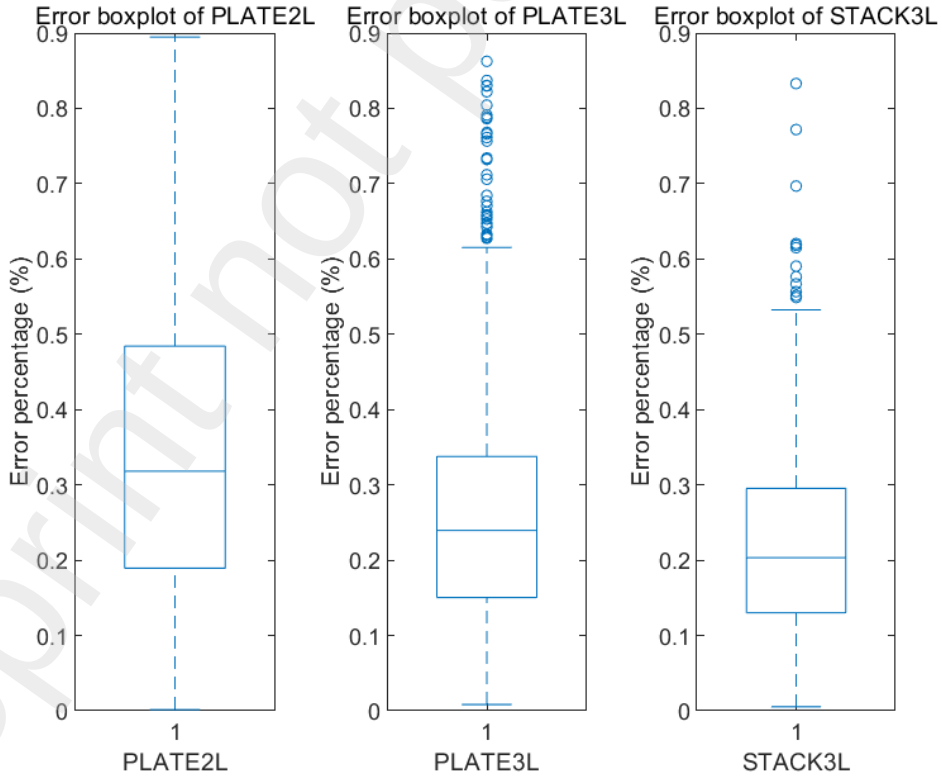

Figure 21. Box plots of error distributions

In terms of execution time, scatter plots of the computation time for all sampling points are

provided in Figures 22 through 24. Statistical results indicate that the average prediction time per sampling point for the three patterns is 16.8053 ms (PLATE2L), 34.4496 ms (PLATE3L), and 34.6405 ms (STACK3L). Overall, the neural network model achieves high prediction accuracy while substantially improving the efficiency of capacitance extraction, making it well suited for rapid scanning applications across large parameter spaces.

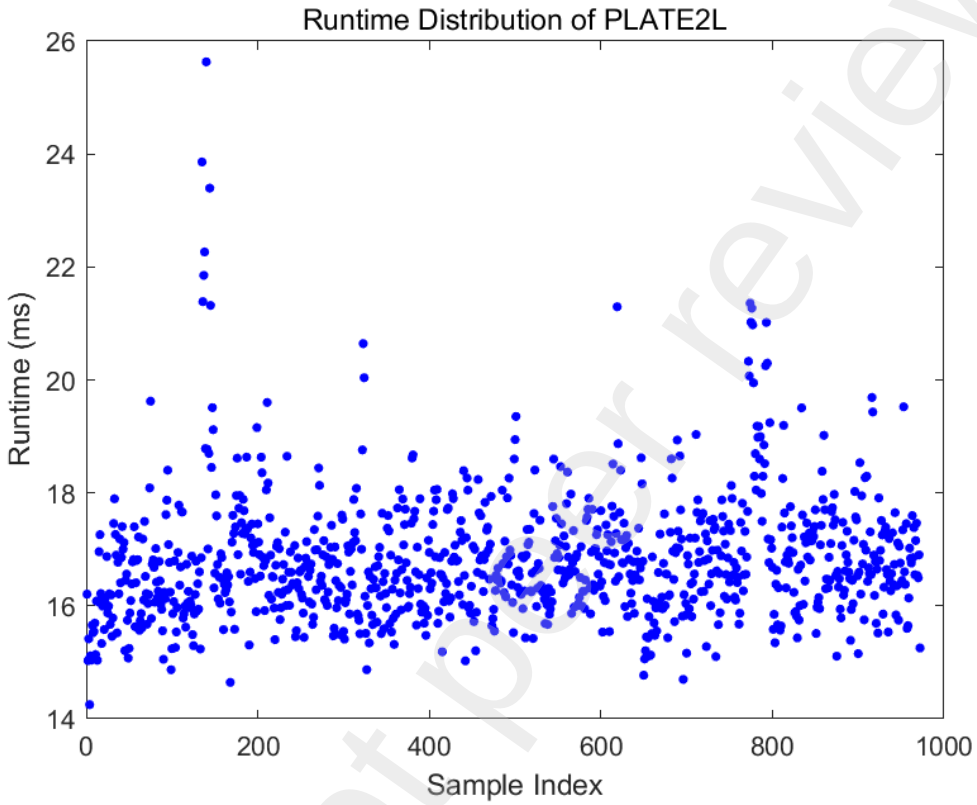

Figure 22. Scatter plot of runtime for the PLATE2L pattern

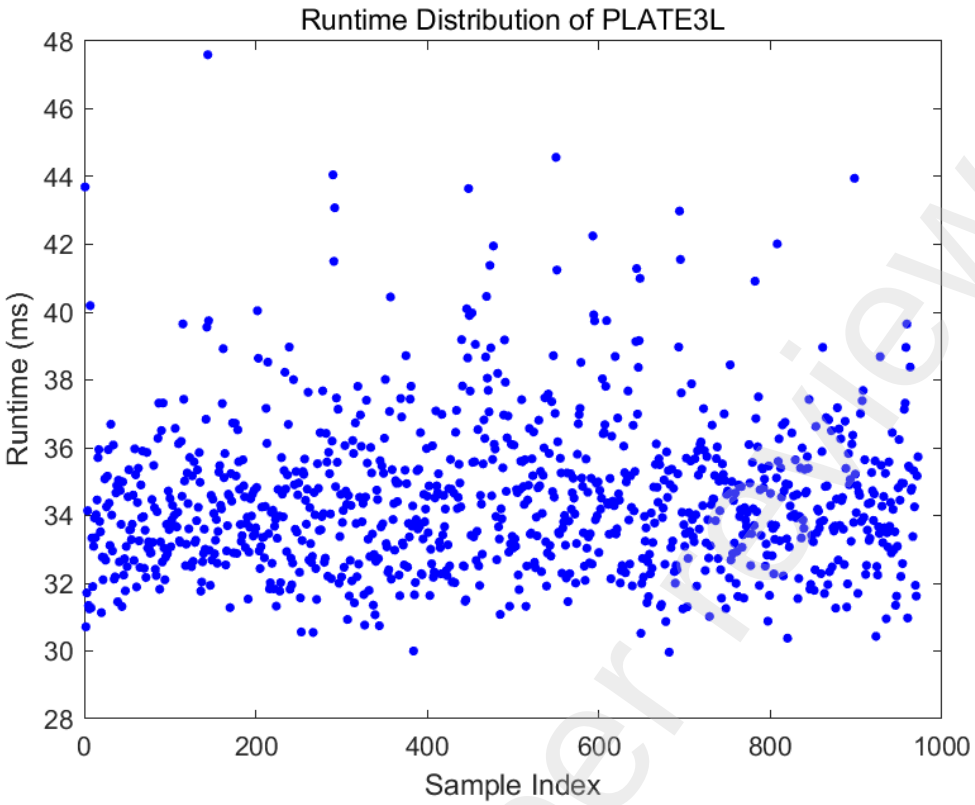

Figure 23. Scatter plot of runtime for the PLATE3L pattern

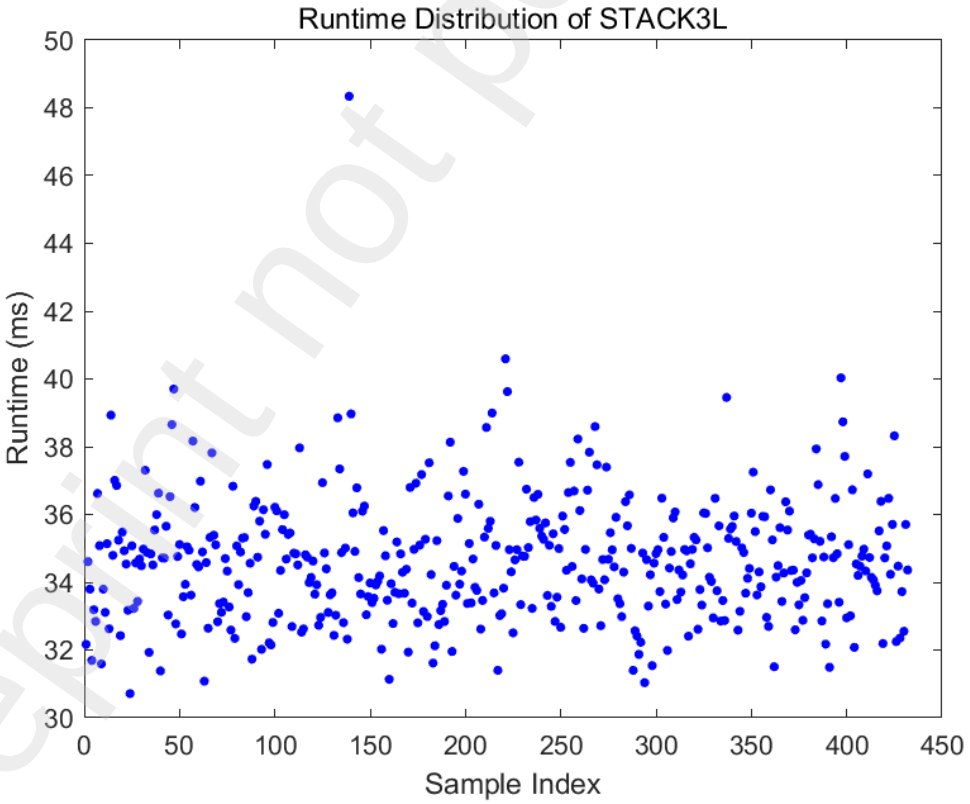

Figure 24. Scatter plot of runtime for the STACK3L pattern

5.Conclusion

This paper has addressed the challenge of fast parasitic capacitance extraction by integrating traditional field solver methods with deep neural networks, establishing a complete workflow encompassing data generation, feature extraction, model training, and performance validation. The main contributions are summarized as follows. First, based on finite element field solver simulations, a comprehensive dataset comprising numerous samples for three typical interconnect structures—PLATE2L, PLATE3L, and STACK3L—was generated, serving as the foundation for neural network training and validation. By appropriately selecting input features, the mapping relationship between structural parameters and capacitance values was effectively established. Second, deep neural network models were designed and trained to accurately learn the complex nonlinear relationships between structural parameters and capacitance values, enabling fast prediction of parasitic capacitance. Extensive testing demonstrates that the prediction errors for all sampling points are within 1%, satisfying the accuracy requirements. Third, comparative analysis against conventional field solvers verifies the advantages of the proposed method in both prediction accuracy and computational speed. Notably, significant time acceleration is achieved in batch predictions over large parameter spaces, offering a new solution for parasitic parameter extraction in integrated circuit design. Finally, additional tests on unseen data further validate the generalization capability and reliability of the trained models, confirming that the proposed approach possesses potential for transfer and broader application.

Future work will focus on continuously improving the computational efficiency and accuracy of the field solver component, as well as introducing emerging network architectures in the deep learning domain to further enhance the accuracy and efficiency of capacitance prediction. These efforts aim to advance parasitic capacitance extraction technology and provide more powerful tools for circuit design and optimization.

Funding: This work was funded by China-Sudan Joint Laboratory of New Photovoltaic Ecological Agriculture (Grant No. 2023YFE0126400), Wuhan Key Research and Development Plan (Grant No. 2024050702030134) and College Student Innovation and Entrepreneurship Training Program (20250100142).

Data Availability Statement: The raw data supporting the conclusions of this article will be made available by the authors on request.

Conflicts: The authors declare no conflict of interest.

References

[1] Nikolaidis S, Chatzigeorgiou A, Kyriakis-Bitzaros E D. Delay and power estimation for a CMOS inverter driving RC interconnect loads [J]. International Symposium on Circuits and Systems (ISCAS), 1998, 6: 368-371.

[2] Tang K T, Friedman E G. Delay and noise estimation of CMOS logic gates driving coupled

resistive–capacitive interconnections [J]. Integration, 2000, 29(2): 131–165.

[3] Martins R, Pyka W, Sabelka R. High-precision interconnect analysis[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 1998, 17(11): 1148–1159.

[4] Campbell J B. Finite difference techniques for ring capacitors [J]. Journal of Engineering Mathematics, 1975, 9(1): 21–28.

[5] Tausch J, White J. A multiscale method for fast capacitance extraction [J]. Proceedings of the 36th Annual ACM/IEEE Design Automation Conference, 1999: 537–542.

[6] Wang C F, Tan KK, Li L W, Kooi P S, Leong M S. Efficient capacitance extraction of MMIC passive components using MoM combined with discrete complex image method [J]. Microwave and Optical Technology Letters, 2002, 32(3): 216–219.

[7] Batterywala S, Ananthakrishna R, Luo Y, Gyure A. A statistical method for fast and accurate capacitance extraction in the presence of floating dummy fills [J]. Proceedings of the 19th International Conference on VLSI Design Held Jointly with the 5th International Conference on Embedded Systems Design (VLSID), 2006: 6–6.

[8] Yu W J, Wang Z Y. Capacitance extraction [J]. Encyclopedia of RF and Microwave Engineering, 2005: 1–12.

[9] Yu W J. The Floating Random Walk Algorithms for Capacitance Extraction Problems in IC and FPD Design [Z]. 2015.

[10] Shomalnasab G, Zhang L. New analytic model of coupling and substrate capacitance in nanometer technologies [J]. IEEE Transactions on Very Large Scale Integration (VLSI) Systems, 2014, 23(7): 1268–1280.

[11] Bansal A, Paul B C, Roy K. An analytical fringe capacitance model for interconnects using conformal mapping [J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2006, 25(12): 2765–2774.

[12] Liang W, Yu W. A 2-D capacitance solver with finite difference method[C]. Proceedings of the 2020 China Semiconductor Technology International Conference (CSTIC), 2020: 1–3.

[13] Ren D, Xu X, Qu H, Ren Z. Two-dimensional parasitic capacitance extraction for integrated circuit with dual discrete geometric methods[J]. Journal of Semiconductors, 2015, 36(4): 045008.

[14] Van der Meijs N P, van Genderen A J. An efficient finite element method for submicron IC capacitance extraction [C]. Proceedings of the 26th ACM/IEEE Design Automation Conference, 1989: 678–681.

[15] Li T, et al. 3D capacitance extraction with the method of moments [D]. Worcester Polytechnic Institute doctoral dissertation, 2010.

[16] Dengi E A, Rohrer R A. Boundary element method macromodels for 2-D hierarchical

capacitance extraction[C]. Proceedings of the 35th Annual Design Automation Conference, 1998: 218–223.

[17] Borkowski M. 2D capacitance extraction with direct boundary methods [J]. Engineering Analysis with Boundary Elements, 2015, 58: 195–201.

[18] Zhai K, Yu W. The 2-D boundary element techniques for capacitance extraction of nanometer VLSI interconnects [J]. International Journal of Numerical Modelling: Electronic Networks, Devices and Fields, 2014, 27(4): 656–668.

[19] Hong W, Sun W K, Zhu Z H, Ji H, Song B, Dai W M. A novel dimension-reduction technique for the capacitance extraction of 3-D VLSI interconnects [J]. IEEE Transactions on Microwave Theory and Techniques, 1998, 46(8): 1037–1044.

[20] Kasai R, Kanamoto T, Imai M, Kurokawa A, Hachiya K. Neural network-based 3D IC interconnect capacitance extraction [C]. Proceedings of the 2019 2nd International Conference on Communication Engineering and Technology (ICCET), 2019: 168–172.

[21] Shao Y S, Clemons J, Venkatesan R, Zimmer B, Fojtik M, Jiang N, Keller B, Klinefelter A, Pinckney N, Raina P, Tell S G. Simba: Scaling deep-learning inference with multi-chip-module-based architecture [C]. Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture, 2019: 14–27.

[22] Tawfik W Z, Mohammad S N, Rahouma K H, Tammam E, Salama G M. An artificial neural network model for capacitance prediction of porous carbon-based supercapacitor electrodes [J]. Journal of Energy Storage, 2023, 73: 108830.

[23] Arora N D, Raol K V, Schumann R, Richardson L M. Modeling and extraction of interconnect capacitances for multilayer VLSI circuits[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 1996, 15.1: 58–67.

[24] Yang D, Yu W, Guo Y, Liang W. CNN-Cap: Effective convolutional neural network based capacitance models for full-chip parasitic extraction [C]. Proceedings of the 2021 IEEE/ACM International Conference on Computer Aided Design (ICCAD), 2021: 1–9.

[25] Huang N K. Implementation of Algorithms to Determine the Capacitance Sensitivity of Interconnect Parasitics in the Magic VLSI Layout Tool [D]. Vancouver: University of British Columbia, 2009.

[26] He K, Zhang X, Ren S, Sun J. Deep residual learning for image recognition [C]. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016: 770-778.

[27] Wang Q, Wang K, Li Q, Yang Z, Jin G, Wang H. MBNN: A multi-branch neural network capable of utilizing industrial sample unbalance for fast inference [J]. IEEE Sensors Journal, 2020, 21(2): 1809-1819.

[28] Pan S Y Q. A survey on transfer learning [J]. IEEE Transactions on Knowledge and Data Engineering, 2010, 22(10): 1345-1359.

[29] Ba J, Frey B. Adaptive dropout for training deep neural networks [C]. In: Advances in Neural Information Processing Systems, 2013: 26.